

21CSE524T COMPUTER VISION TECHNIQUES

Unit -1

Authour
Dr.M.Meenakshi,
Assistant Professor ,
Department of Computational Intelligence,
SRM Institute of Science and Technology,
Kattankulathur.

Topics in Unit 1

- *Introduction*
- *Review of image processing techniques*
- *Classical filtering operations*
- *Thresholding techniques*
- *Edge detection techniques*
- *Corner and interest point detection*

Introduction to Computer Vision

- **Computer Vision (CV)** is basically **teaching computers to “see” and understand images and videos**, like humans do.
- Think of it like this:
 - 👁️ Humans → eyes + brain → understand what we see
 - 💻 Computer → camera + algorithms → understand what it sees
- ◆ **What does it actually do?**
 - It helps computers answer questions like:
 - What is in this image?
 - Is this a cat or a dog?
 - Where is the face?
 - Is this person wearing a helmet?
 - Is this tumor visible in the scan?

Common tasks in Computer Vision

- **Image Classification**
→ “What is this image?”
Example: Cat / Dog / Car
- **Object Detection**
→ “Where is the object?”
Example: Detect people or vehicles with boxes
- **Image Segmentation**
→ “Which pixels belong to which object?”
Example: Separate road, car, sky
- **Face Recognition**
→ “Whose face is this?”
- **Motion Tracking**
→ “Where is the object moving?”

Introduction to Computer Vision

- Field of artificial intelligence that enables computers to derive meaningful information from digital images, videos, and other visual inputs, and take actions or make recommendations based on that information.



Importance of Computer Vision and Applications

- Automation and Efficiency- Industry Automation



Importance of Computer Vision and Applications

- Enhanced Safety and Security- Surveillance, Autonomous vehicle



Importance of Computer Vision and Applications

- Healthcare Advancements- Disease Diagnosis, Surgical Assistance

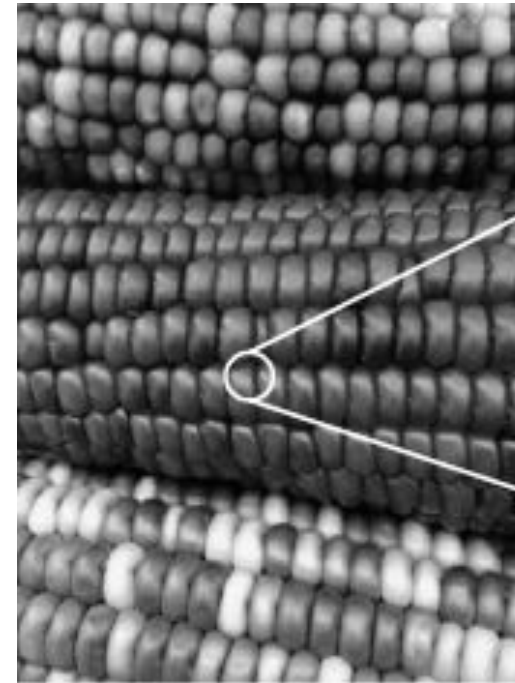
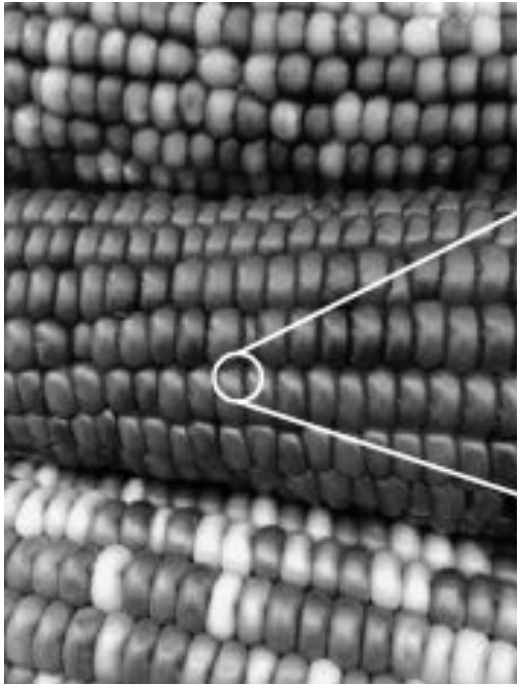


Importance of Computer Vision and Applications

- Improved User Experiences-Augmented Reality, Facial recognition



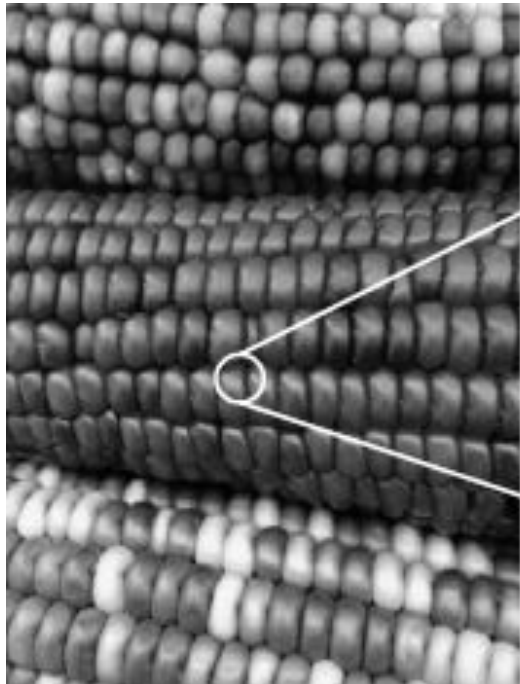
Image Representation



75	76	70	51
132	115	92	61
177	163	133	92
174	180	158	120
139	153	152	136

Data Matrix

Image Representation



$$f(x,y) = \begin{bmatrix} f(0,0) & f(0,1) & f(0,2) & f(0,3) & f(0,4) & \dots \\ f(1,0) & f(1,1) & f(1,2) & f(1,3) & f(1,4) & \dots \\ f(2,0) & f(2,1) & f(2,2) & f(2,3) & f(2,4) & \dots \\ f(3,0) & f(3,1) & f(3,2) & f(3,3) & f(3,4) & \dots \\ | & | & | & | & | & \dots \\ | & | & | & | & | & \dots \end{bmatrix}$$

Image Representation - Gray

- Image Representation
- 2 dimensional function $f(x, y)$
- X, Y are spatial Co-ordinates
- The amplitude of 'f' is called intensity or grey level at the point (x,y)
- Finite elements called pixels which has location and value
- Pixel is stored in a single byte(8 bits) of memory



75	76	70	51
132	115	92	61
177	163	133	92
174	180	158	120
139	153	152	136

Data Matrix

- 256 (0-255) p
ls

$$2^8 = 256$$

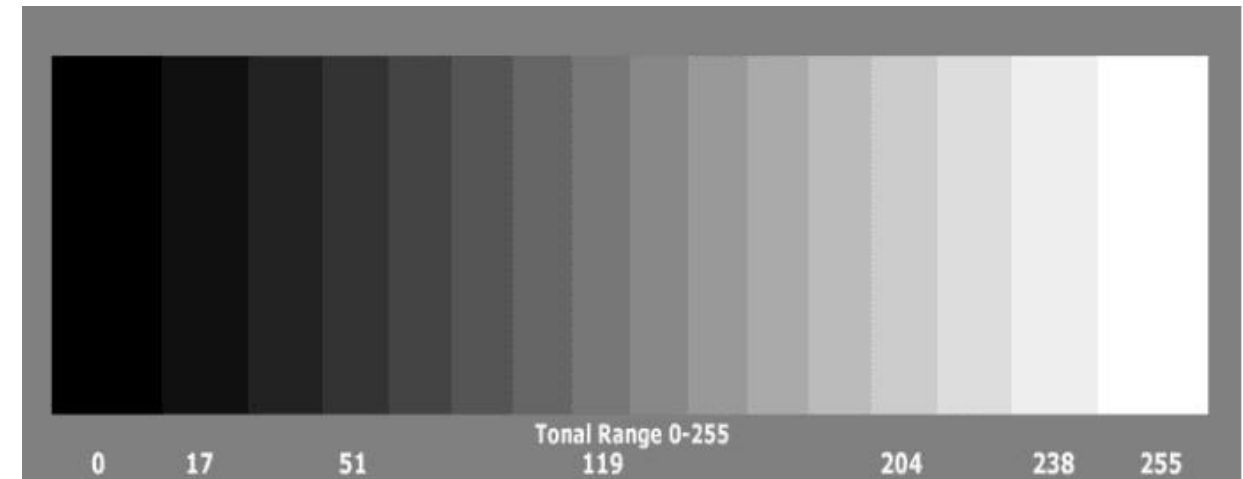


Image Representation- Colour

Colour images are made up of pixels which holds 3 numbers corresponding to RGB (Red, Green, Blue).

$Fr(x,y)$, $Fg(x,y)$, $Fb(x,y)$

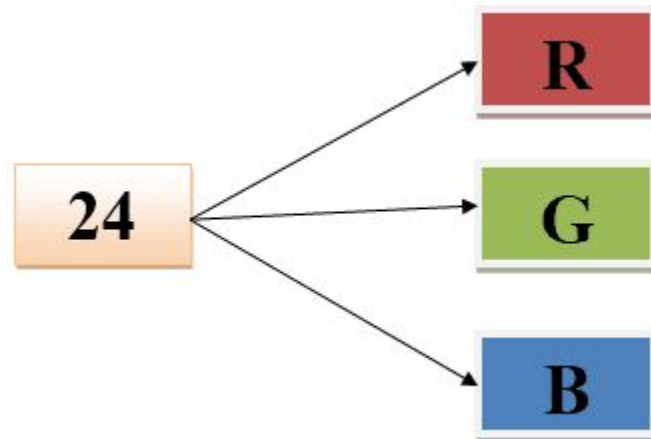


Fig: 16,777,216 colors



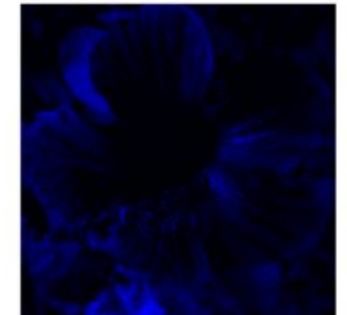
R (Red)



G (green)



B (blue)



Possible Colours:

$$2^8 \text{ (of Red)} \times 2^8 \text{ (of Green)} \times 2^8 \text{ (of Blue)} =$$

$$2^{24} = 16,777,216$$

Review of image processing techniques

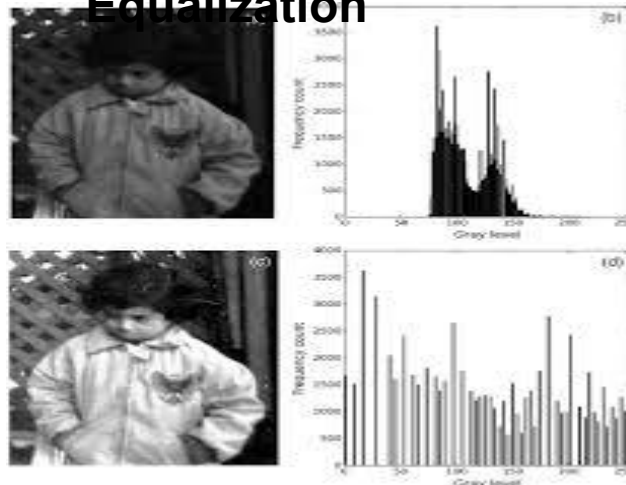


Basic Image Processing Techniques

Grayscale Conversion



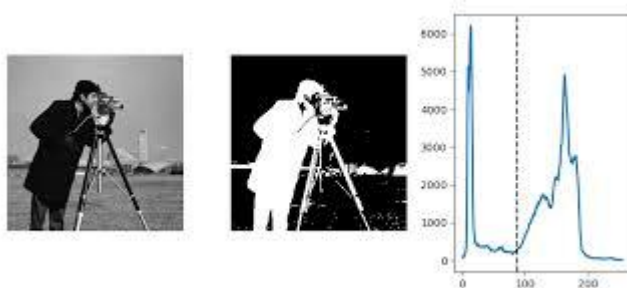
Histogram Equalization



Smoothing/Blurring



Thresholding



Edge Detection

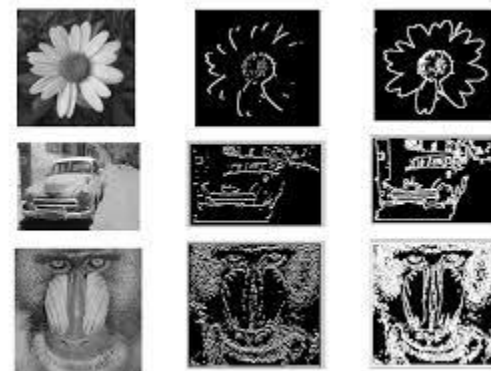


Image Representation - Gray

◆ 1. Grayscale Conversion

What it does

Converts **RGB (3 channels)** → **Gray (1 channel)**

Why we use it

Removes unnecessary color info

Reduces computation

Faster processing

Helps CV how?

Edges, shapes, textures don't need color

Easier for:

- Face detection

- OCR

- Edge detection

Example

Instead of:

(255, 0, 0) red pixel

We get:

120 gray value

👉 Less data = faster model

Image Representation - Gray

2. Histogram Equalization

What it does

Improves **contrast** of image

Problem

Some images are:

too dark OR

too bright OR

low contrast

Solution

Spreads pixel intensity values evenly

Helps CV how?

Objects become clearer

Features easier to detect

Improves:

- Face recognition

- Medical imaging

- Night vision

Example

Dark face → clearer face → better detection

Image Representation

3. Smoothing / Blurring

What it does

Removes **noise**

Noise = random dots/grain

Causes false edges and wrong predictions

Helps CV how?

Reduces unwanted details

Makes shapes clean

Improves:

- Edge detection

- Segmentation

- Object detection

Filters used

Gaussian blur

Mean filter

Median filter



Clean image = better accuracy

Image Representation

4. Thresholding

What it does

Converts image → **Binary (0 or 255)**

Why

- Separates:
- Object
- Background

Helps CV how?

Easy to:

- count objects
- detect shapes
- detect text

Example

White object on black background

Perfect for:

OCR

Fingerprint detection

Document scanning

Image Representation

5. Edge Detection

What it does

Finds **boundaries of objects**

Why

Edges = most important features

Helps CV how?

Used for:

- Shape detection
- Lane detection (self-driving cars )
- Object tracking
- Face recognition

Methods

- Sobel
- Canny
- Laplacian

 Model focuses only on structure, not useless pixels

Basic Image Processing Techniques-Geometric Transformations

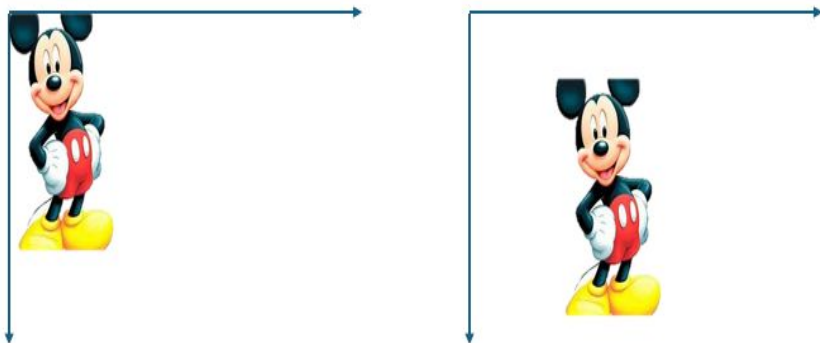
Scaling



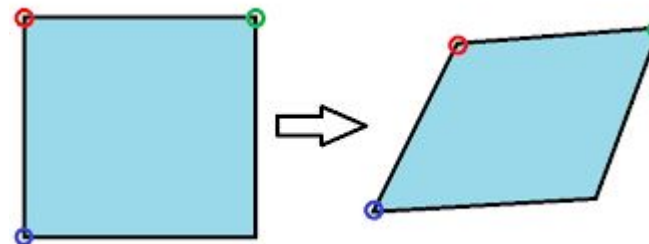
Rotation



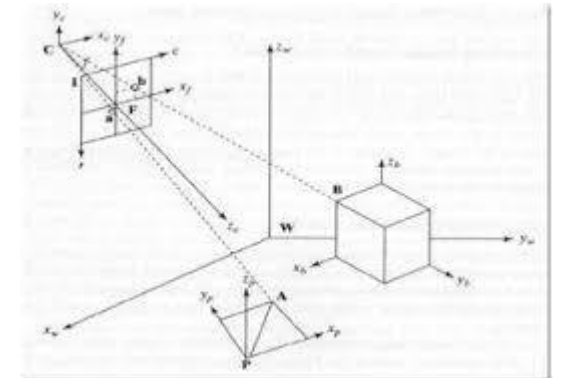
Translation



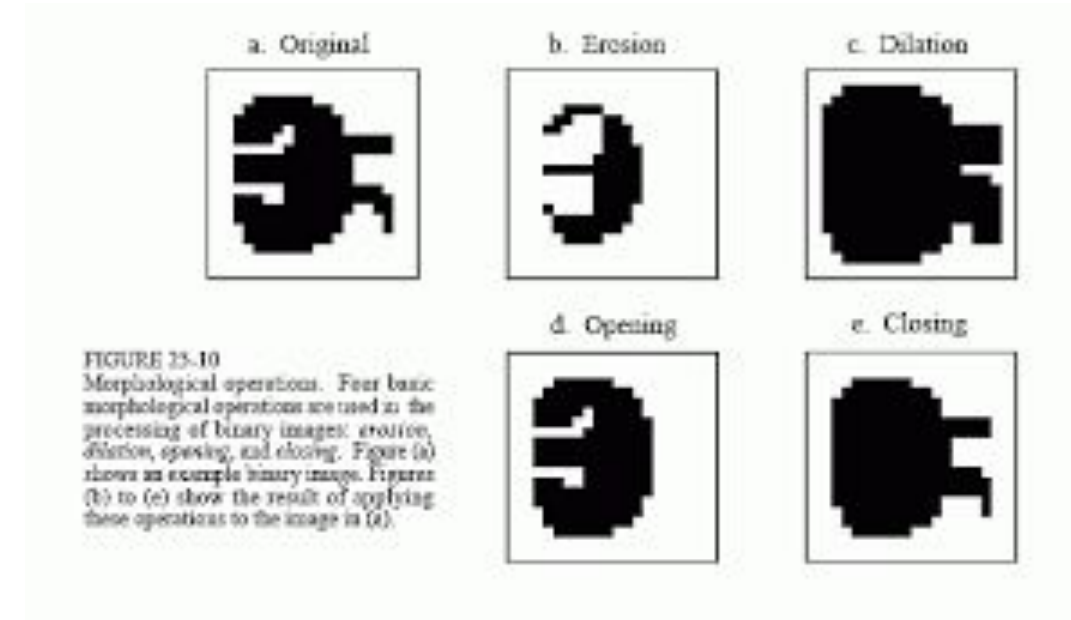
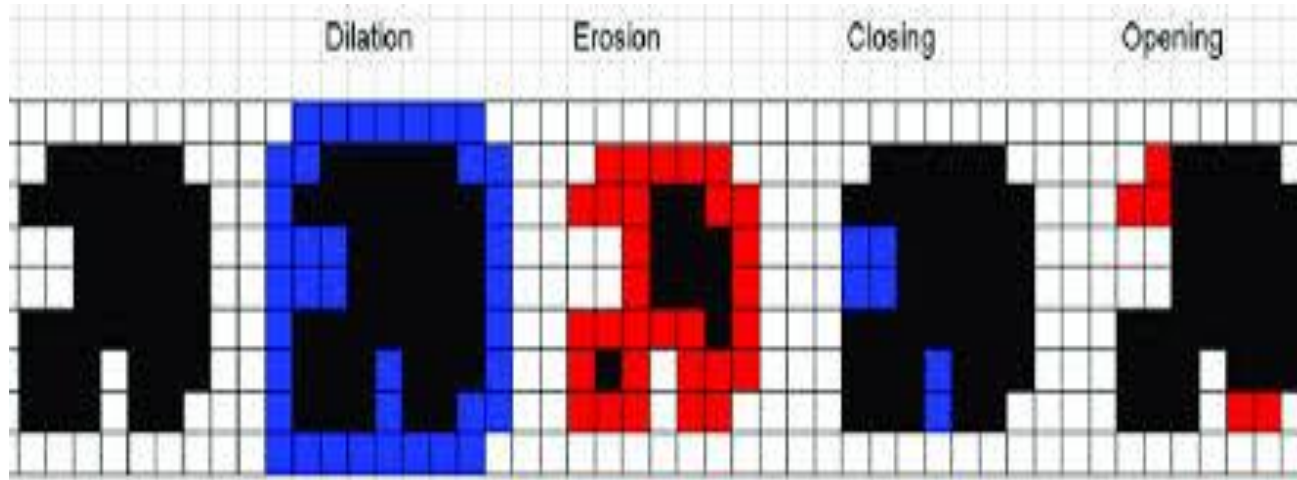
Affine Transform



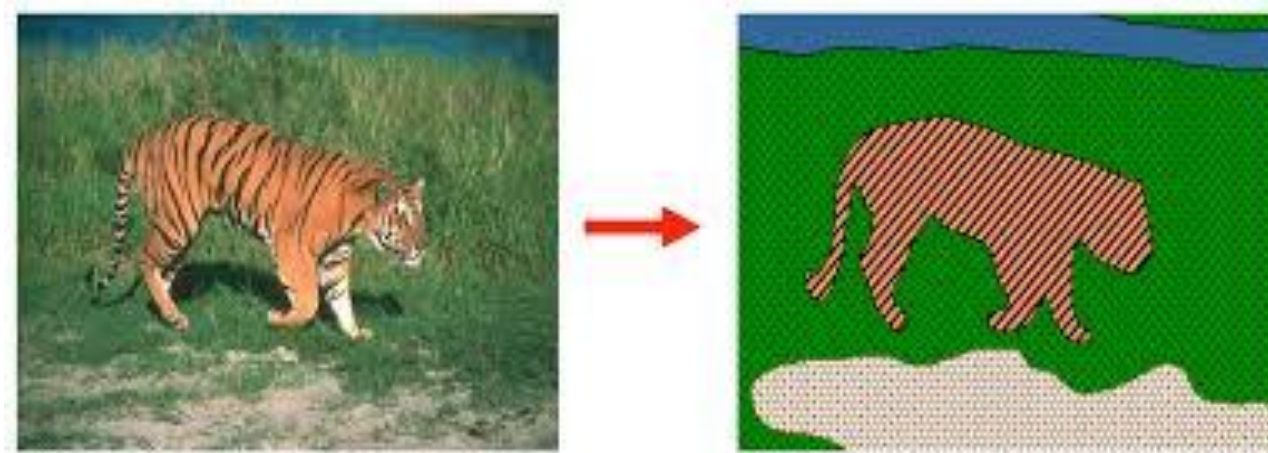
Perspective Transform



Basic Image Processing Techniques- Morphological Operations



Basic Image Processing Techniques- Segmentation



Basic Image Processing Techniques- Segmentation

★ What is Segmentation?

Simple meaning

Segmentation = dividing an image into meaningful parts (regions/objects).

Instead of:

✗ Whole image

We get:

✓ Object separated from background

🧠 **Super easy idea**

Think like this:

If I ask computer:

👉 “Where is the tiger?”

It must:

Find tiger pixels

Separate them from grass/sky/ground

That process = **Segmentation**

Basic Image Processing Techniques- Segmentation

Why do we need segmentation?

Without segmentation:

Computer sees only pixels

With segmentation:

Computer understands objects

So it helps in:

- ✓ object detection
- ✓ counting objects
- ✓ tracking
- ✓ measurement
- ✓ recognition

Types of Segmentation

1. Thresholding (Basic)

Separate based on intensity

Example:

pixel $> 128 \rightarrow$ white

pixel $< 128 \rightarrow$ black

Used in:

- document scanning
- OCR
- Simple but limited

2. Edge-based Segmentation

Use edges to find boundaries

Steps:

- detect edges
- connect edges
- form object

Used in:

- shape detection

Problem: noisy sometimes

Types of Segmentation

3. Region-based Segmentation

Group similar pixels

Based on:

- color
- texture
- intensity

Example:

All green pixels → grass region

Used in:

- medical imaging
- satellite images

4. Clustering (K-means)

ML method

Groups pixels automatically

Example:

$K = 3 \rightarrow$ sky, grass, tiger

Very popular method

Types of Segmentation

5. Deep Learning Segmentation (Modern 🔥)

CNN models

Types:

- Semantic segmentation (class-wise)
- Instance segmentation (object-wise)

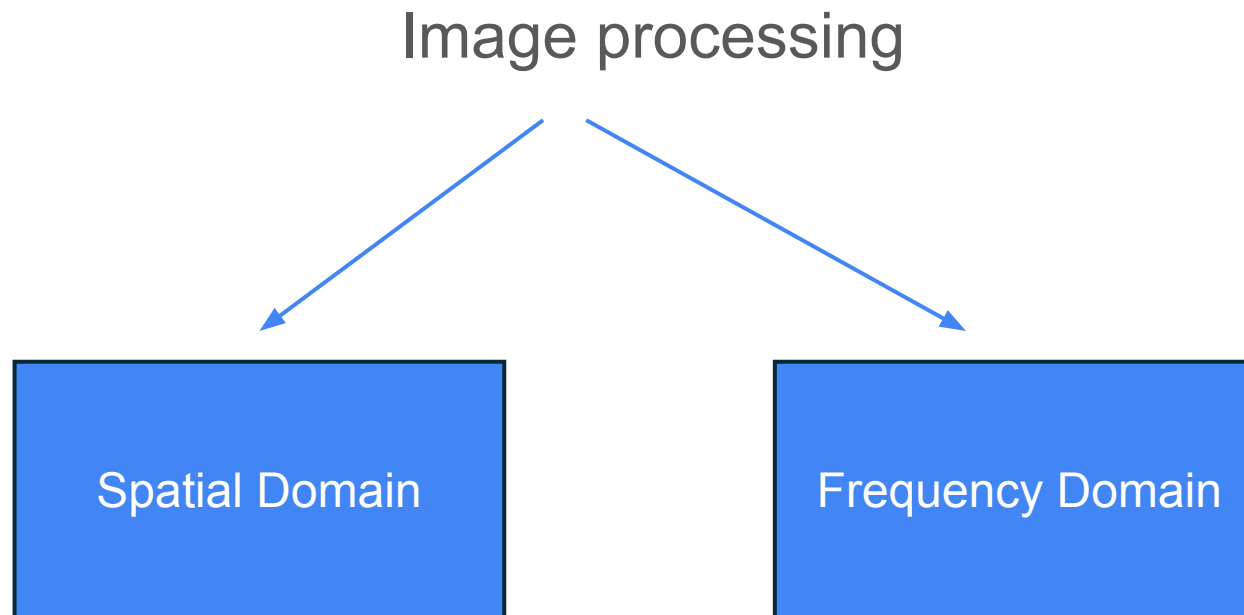
Models:

- U-Net
- Mask R-CNN
- YOLO-seg

Used in:

- self-driving cars
- medical tumor detection
- robotics

Image processing methods



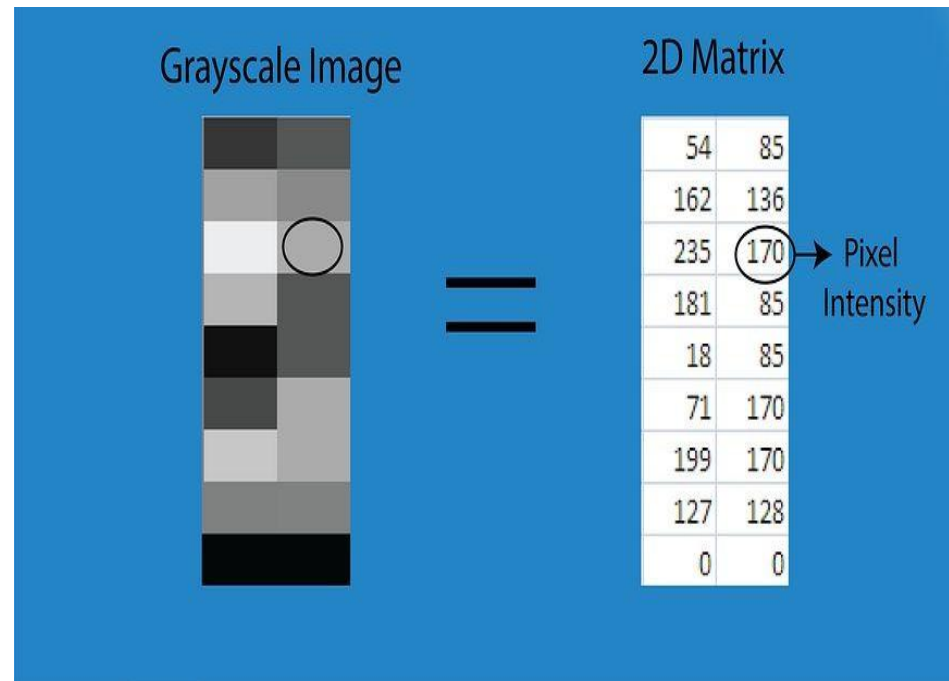
Spatial domain

- Spatial domain refers to the **image plane itself**, and techniques involve the **direct manipulation of pixel values** within the image.
- These methods operate on **individual pixel values** and **their neighboring pixels**.

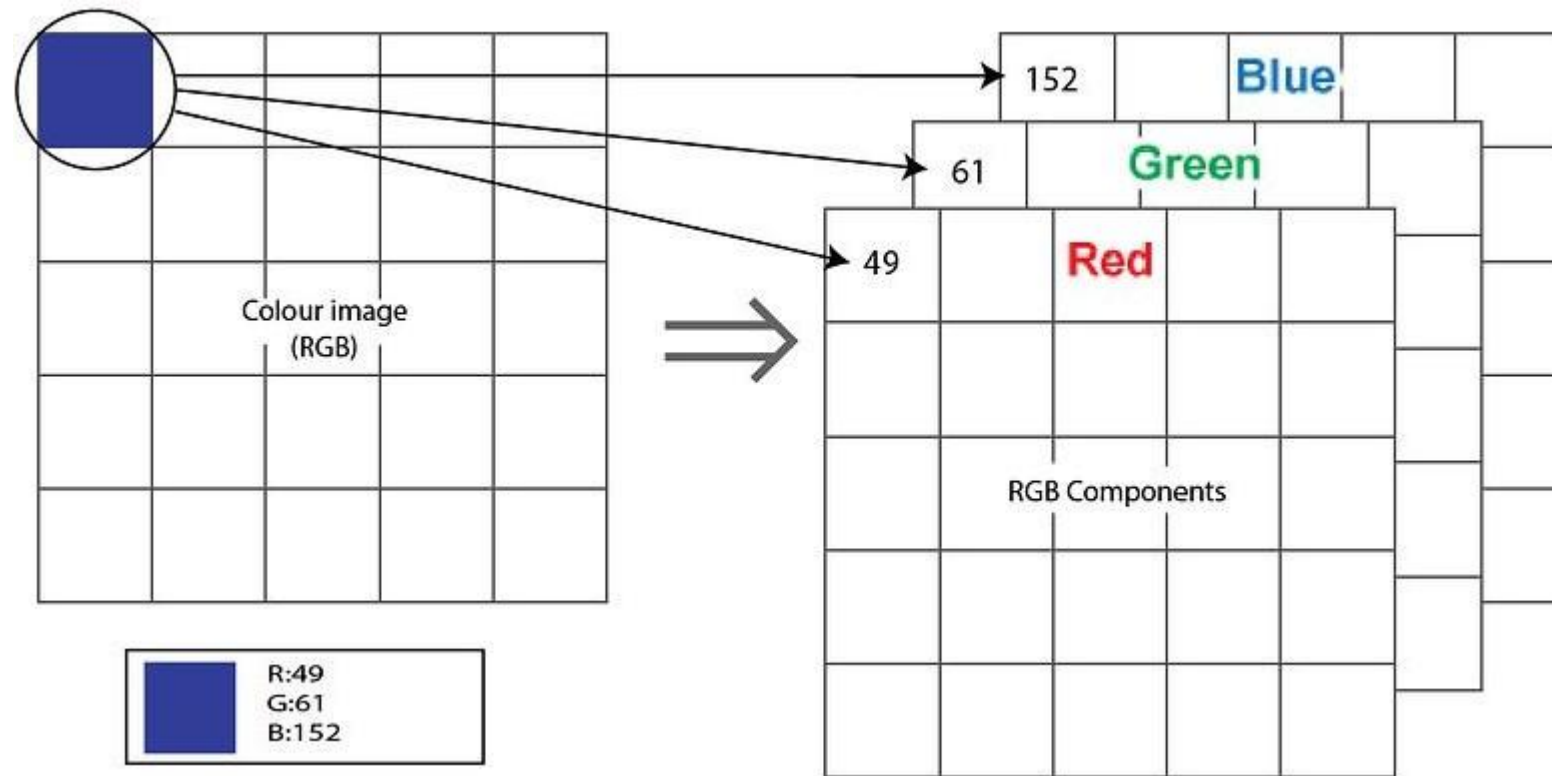


Spatial Domain For Gray scale image

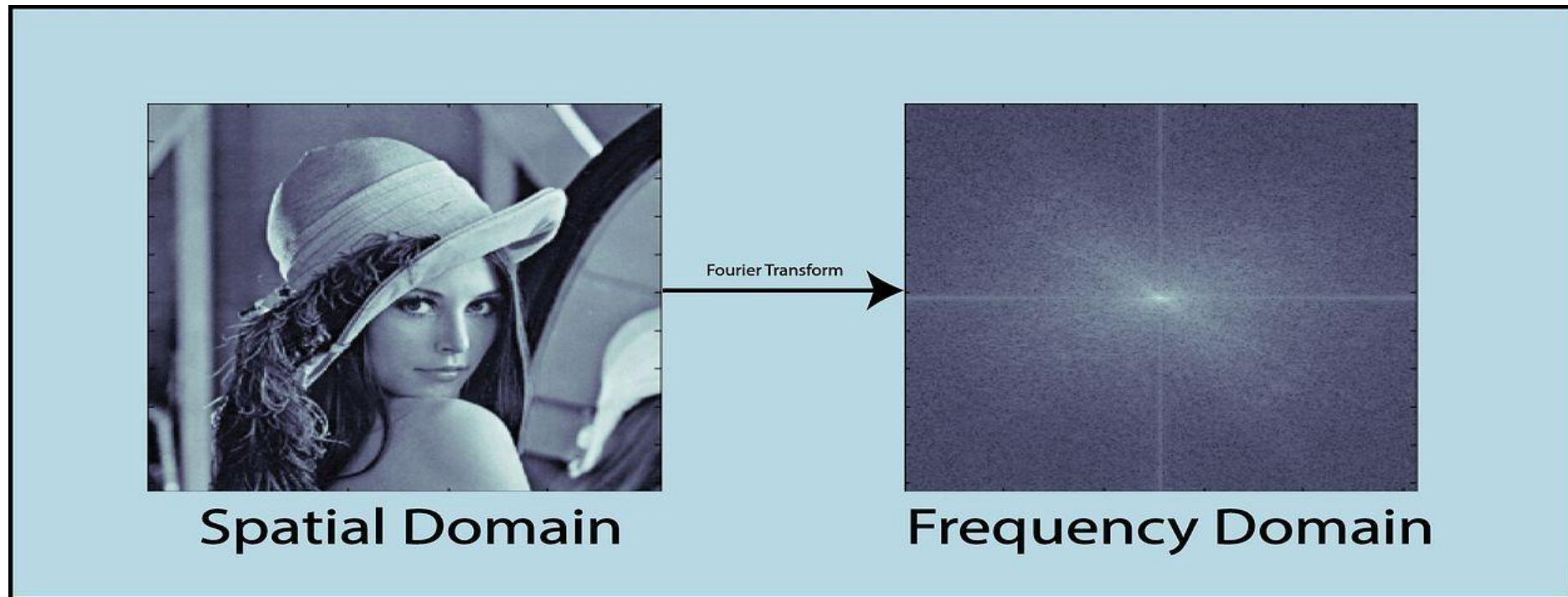
- An image can be represented in the form of a 2D matrix where each element of the matrix represents **pixel intensity**.



Spatial Domain For Color Image



Frequency Domain



Mathematical Model of Spatial domain

- Spatial domain processes will be denoted by the expression

$$g(x, y) = T[f(x, y)]$$

- where **$f(x, y)$** is the input image
- **T** is an operator on f (Different operations are carried out in this value) defined over some neighborhood of (x, y) .
- **$g(x, y)$** is the processed image

Types of Spatial domain operations

. **Intensity Transformation / Point Operations**

- Operates on single pixel of an image.
- Applying the **same transformation** to each pixel in a grayscale image, based on its original pixel value and independent of its location or neighboring pixels.

. **Spatial Filtering**

- This deals with performing operations such as image sharpening, by working in a neighborhood of every pixel in an image.
- The output value of these operations is dependent on the values of the function $f(x,y)$ and its neighborhood.

Intensity Transformation / Point Operations

- Spatial operation

$$g(x, y) = T[f(x, y)]$$

- Transformation Function of point processing

$$s = T(r) \rightarrow \text{Transformation function}$$

- where r is the pixels of the input image and s is the pixels of the output image.
- T is a transformation function that maps each value of r to each value of s .

Point Processing techniques

- Image Negation (Inversion)
- Log Transformation
- Power-Law (Gamma) Transformation
- Contrast Stretching (Normalization)
- Thresholding (Binary, Adaptive)
- Histogram Equalization
- Histogram Matching (Specification)

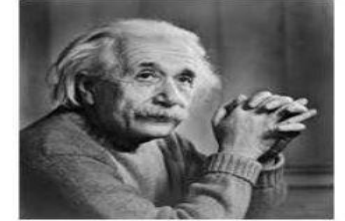
Image Negation (Inversion)

- In negative transformation, each value of the input image is subtracted from the $L-1$ and mapped onto the output image.

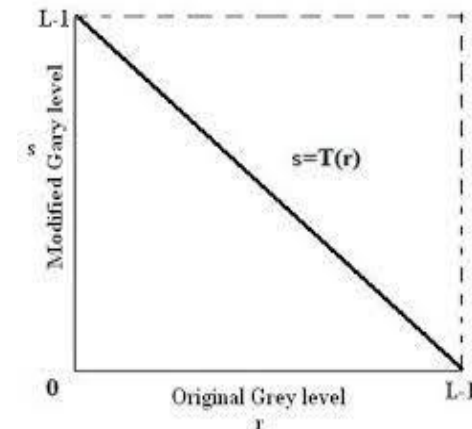
$$s = (L - 1) - r$$

- $L \rightarrow$ the maximum intensity level of pixel values in the image.
- L is typically 256, representing pixel values from 0 (black) to 255 (white) in gray scale.
- $r \rightarrow$ original pixel value of an input image before transformation

Input Image



Output Image



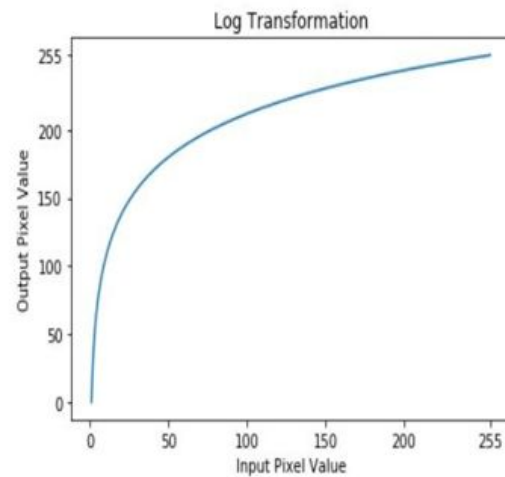
Log Transformation

- Logarithmic transformation is divided into **two types**:
 1. Log transformation - > Replacing each pixel value with its log value
 2. Inverse log transformation -> Finding inverse of log transformation

Log Transformation

- Logarithmic transformation: **$s = c \log(r + 1)$**
- **s and r** $\rightarrow$ pixel values for input and output image. And **c is constant**.
In the formula, we can see that 1 is added to each pixel value this is because if pixel intensity is zero in the image then $\log(0)$ is infinity so, to have minimum value one is added.
- Application: To **increase the visibility of details in darker areas** and **reducing the overall brightness of the image by** Expands the dark pixels in the image while compressing the brighter pixels

Log Transformation

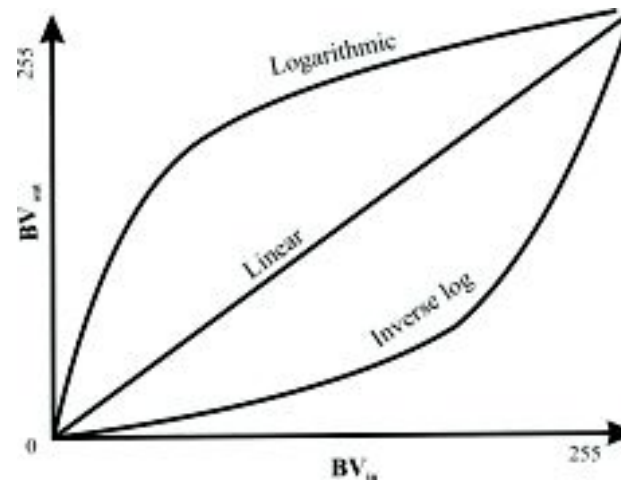


X	0	5	10	20	100	125	127	200	225
y	-	0.69	1	1.3	2	2.09	2	2.3	2.35

*

Inverse Log transformation

- The inverse log transform is opposite to log transform.
$$s = \text{power}(10, r * c) - 1$$
- It maps a narrow range of high gray levels to a much wider range.
- **Application:** The inverse log transform expands the values of light-level pixels while compressing the darker-level values.



Gamma transformation

- The Gamma (γ) or commonly used image enhancement technique that adjusts the intensity values of an image based on a power function.
- This transformation can be applied to **change the overall brightness / visibility / contrast of an image.**
- The main benefit of using Gamma is the more natural look of images and the **ability to differentiate between dark tones in a darker image.**

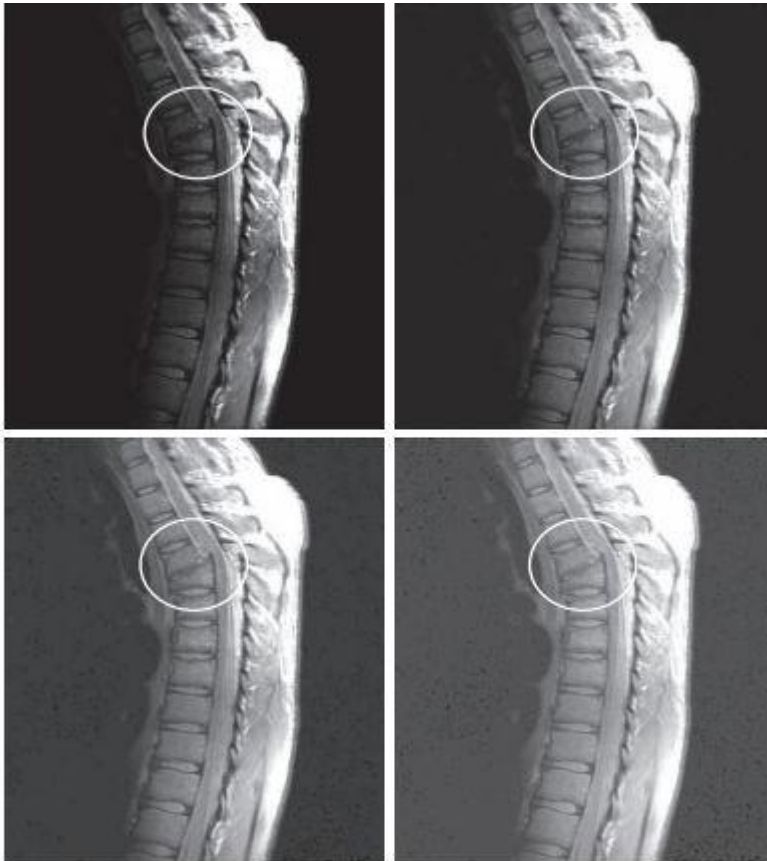
Gamma transformation

$$S = Cr^\gamma$$

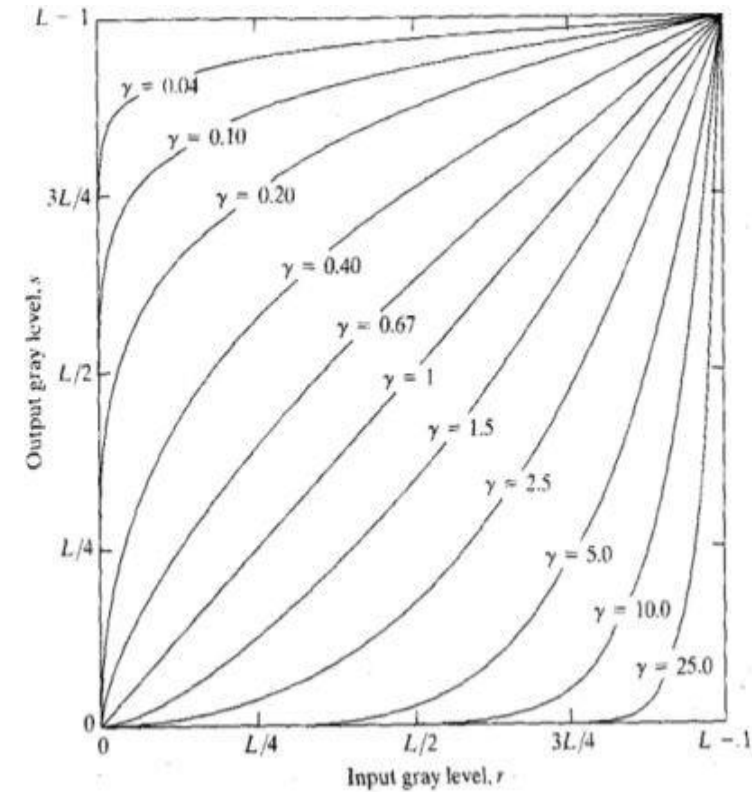
- ▶ $\gamma < 1$: changes darker to brighter images
- ▶ $\gamma > 1$: changes brighter to darker images

- s: This typically represents the output signal or value.
- c: This is a constant factor that scales the input value.
- r: This is the input variable or value.
- γ (gamma): This is the **exponent to which the input value (r) is raised**.
- Here, γ is gamma, by which this **transformation factor** is known as gamma transformation.
- Variation in the value of γ varies the enhancement of the images

Gamma transformation



Region of the fracture is enclosed by the circle: $c = 1$, $\gamma = 0.6, 0.4, 0.3$

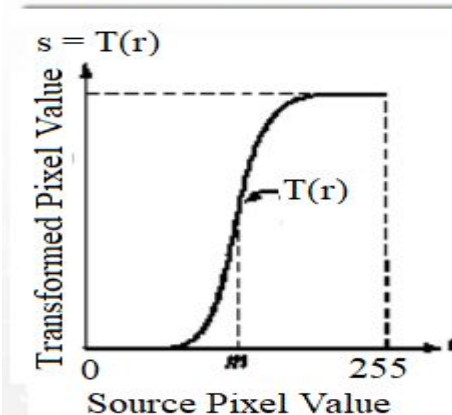


Plots of the equation $s = cr^\gamma$ for various values of γ ($c=1$ in all cases)

Contrast stretching

- Contrast stretching is used to increase the dynamic range of the gray levels in the image.
- For example, in an 8-bit system the image display can show a maximum of 256 gray levels.
- **Contrast stretching**, involves compressing pixel values below a threshold "m" into a narrower range, which makes them appear darker and pixel values below a threshold "m" into a broader range.

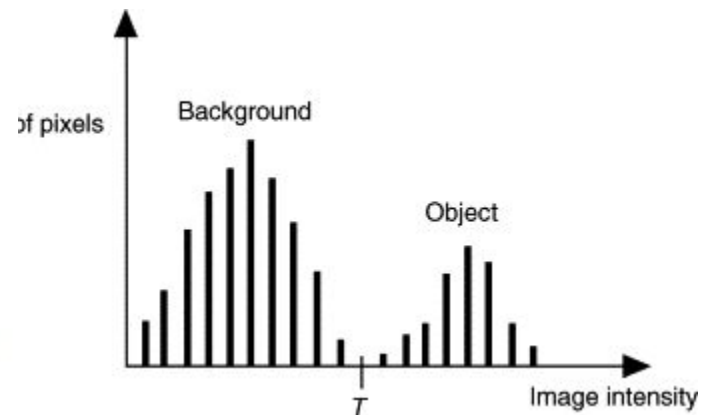
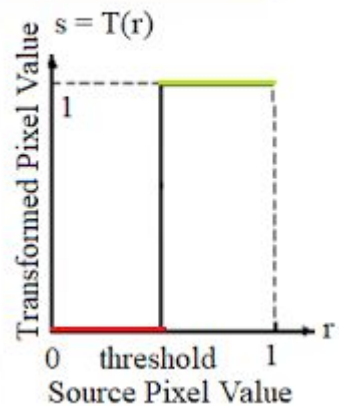
Contrast Stretching



Thresholding

Image thresholding is a technique that simplifies a grayscale image into a binary image by classifying each pixel value as either black or white based on its intensity level or gray-level compared to the threshold value.

Thresholding



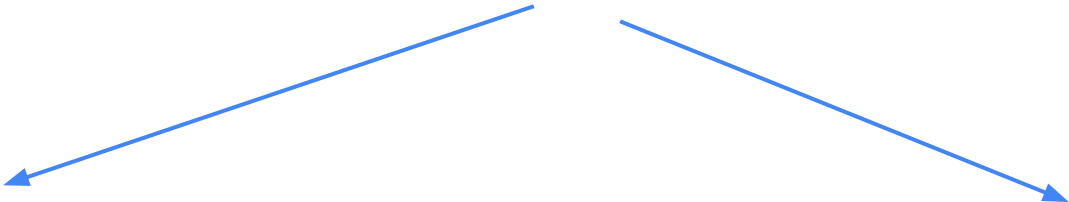
Original Image



Thresholded and segmented Image

Spatial Filter

Types of Spatial Filter



Smoothing filter(Low pass filter)

Linear Filter (Mean Filter)

Gaussian filter

Order Statistics (Non-linear) filter

Sharpening filter (High pass filter)

Linear Filter (High pass)

Linear filtering

- Replace each pixel with a linear combination of its neighbors.
- **Convolution kernel:** prescription for the linear combination

10	5	3
4	5	1
1	1	7

 $*$

0	0	0
0	0.5	0
0	1.0	0.5

 $=$

	7	

kernel

- Linear Filtering of an image f of size $M \times N$ filter mask of size $m \times n$ is given by the expression

$$g(x, y) = \sum_{t=-a}^a \left[\sum_{t=-b}^b w(s, t) f(x+s, y+t) \right]$$

where $a = (m-1)/2$ and $b = (n-1)/2$

To generate a complete filtered image this equation must be applied on all pixels of the image.

Linear Filter (Mean Filter)

$$\frac{1}{9} \times$$

1	1	1
1	1	1
1	1	1

Standard average

Box filter

$$\frac{1}{16} \times$$

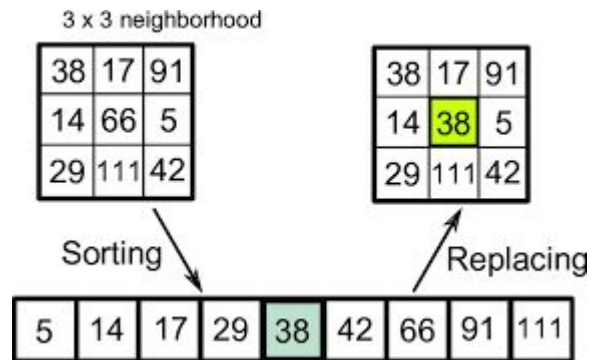
1	2	1
2	4	2
1	2	1

Weighted average

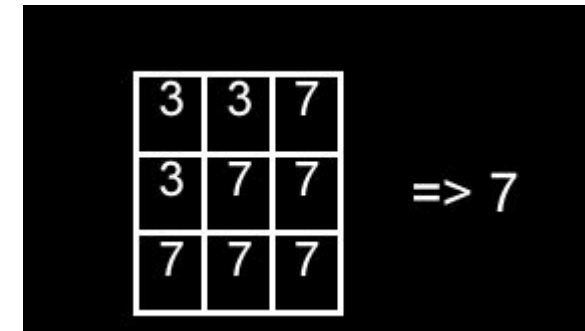
Gaussian filter

Order Statistics filter

- Median filter



Mode filter



Sharpening Filter

Laplacian Filter

0	1	0
1	-4	1
0	1	0

(a)

1	1	1
1	-8	1
1	1	1

(b)

Sharpening Filter (High pass filter)

Laplacian Filter

0	1	0
1	-4	1
0	1	0

(a)

1	1	1
1	-8	1
1	1	1

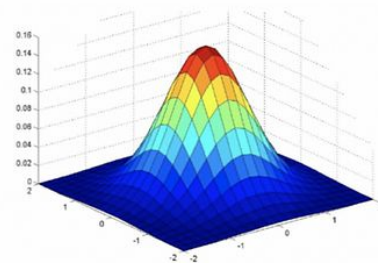
(b)

Laplacian Gaussian Filter

$$LoG = -\frac{1}{\pi\sigma^4} \left[1 - \frac{x^2 + y^2}{2\sigma^2} \right] e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

0	-1	0
-1	4	-1
0	-1	0

Laplacian Filter



Gaussian kernel

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2 + y^2)}{2\sigma^2}}$$

Gaussian Filter

=

0	1	1	2	2	2	1	1	0
1	2	4	5	5	5	4	2	1
1	4	5	3	0	3	5	4	1
2	5	3	-12	-24	-12	3	5	2
2	5	0	-24	-40	-24	0	5	2
2	5	3	-12	-24	-12	3	5	2
1	4	5	3	0	3	5	4	1
1	2	4	5	5	5	4	2	1
0	1	1	2	2	2	1	1	0

LoG Filter

Kernels and Convolutions : High pass filter

- **High-Pass Kernel Filter**
- A basic high-pass filter can be created using a simple kernel that **highlights the center pixel and subtracts the average of the surrounding pixels.**

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Kernels and Convolutions : High pass filter

- **Sobel Filter**

- The Sobel filter uses the **first derivative** to emphasize edges. It calculates the gradient in both the x and y directions and combines them to find the **edge** magnitude.

- Sobel X:

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

- Sobel Y:

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Kernels and Convolutions : High pass filter

- **Prewitt Filter**

- The Prewitt filter is similar to the Sobel filter but uses a simpler gradient approximation.

- Prewitt X:

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

- Prewitt Y:

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Kernels and Convolutions : High pass filter

- **Roberts Cross Operator**

- The Roberts Cross operator performs a simple, quick computation of the gradient and is used for edge detection with a focus on the diagonal edges.

- Roberts X:

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

- Roberts Y:

$$\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

Histogram

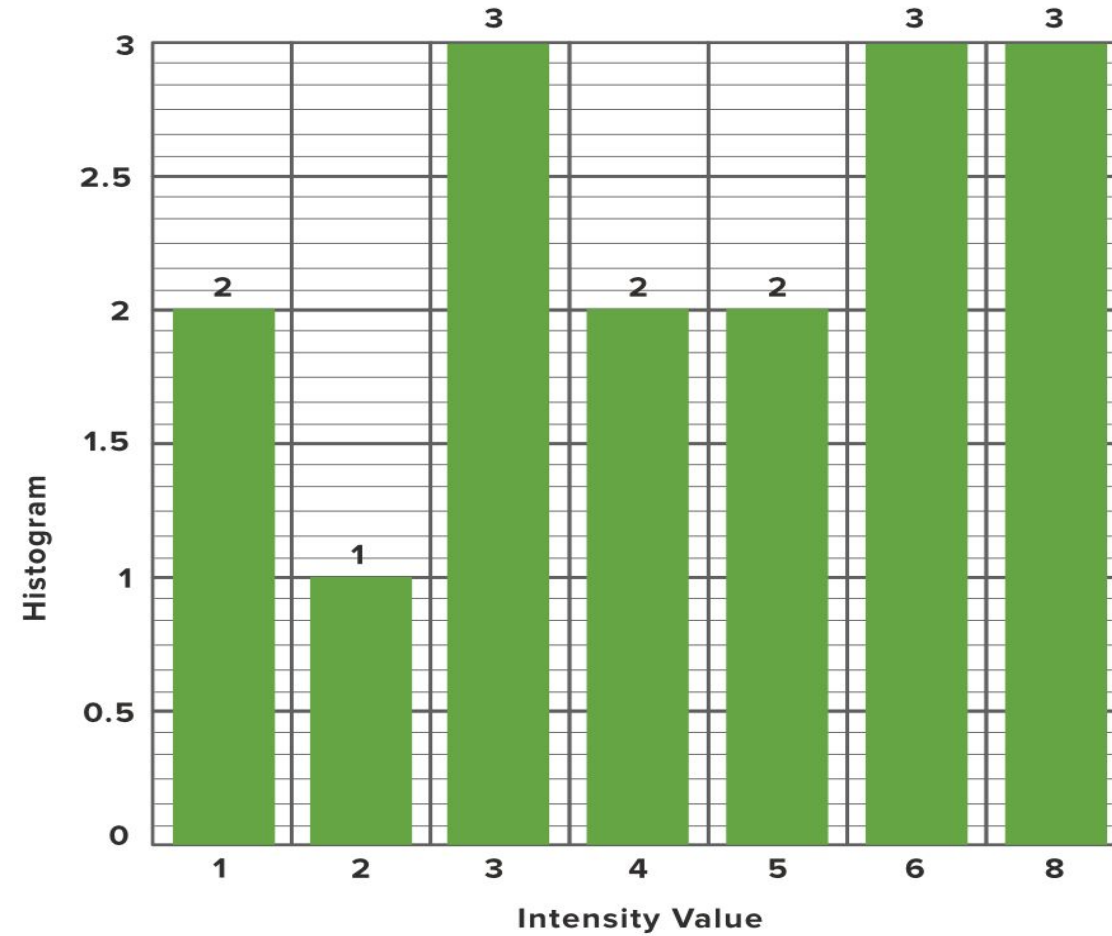
The histogram of a digital image with gray levels in the range $[0, L-1]$ is a discrete function.

$$H(r_k) = n_k$$

- Histogram of an image provides a global description of the appearance of an image.
- Histogram of an image represents the relative frequency of occurrence of various gray levels in an image.

Histogram

3	6	6	8
5	3	1	4
8	6	5	1
4	8	2	3

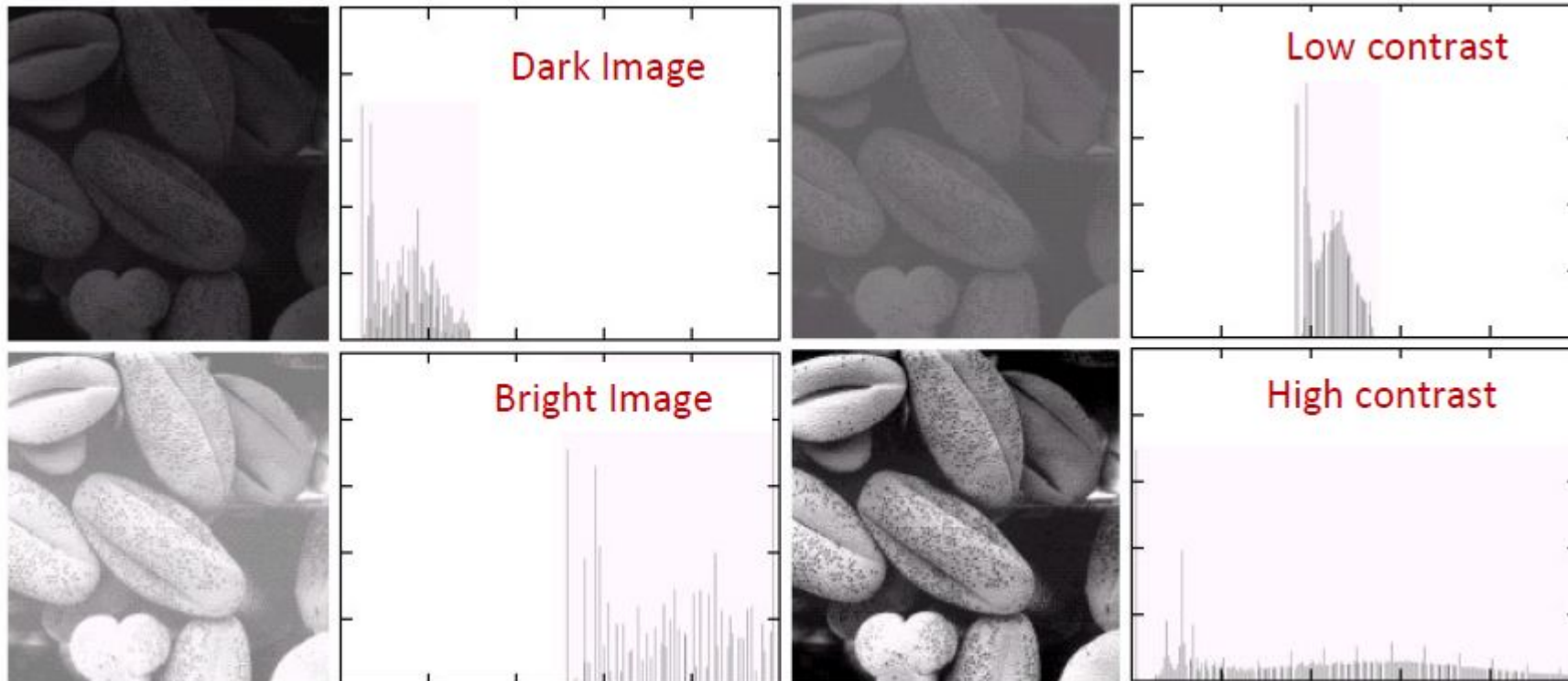


- we can create a histogram graph for each pixel and corresponding occurrence probability.

$$P(r_k) = \frac{n_k}{n}$$

Intensity Value/ Gray Level	#/Occurrence of intensity value/ number of pixels	Probability
1	2	0.125
2	1	0.0625
3	3	0.1875
4	2	0.125
5	2	0.125
6	3	0.1875
8	3	0.1875

Histogram of different images



Histogram Equalisation

Compute the histogram of the image.

Let n_i be the number of pixels with intensity i , and N be the total number of pixels in the image.

The probability density function (PDF) for intensity level i is:

$$p(i) = \frac{n_i}{N}$$

Compute the CDF of the histogram.

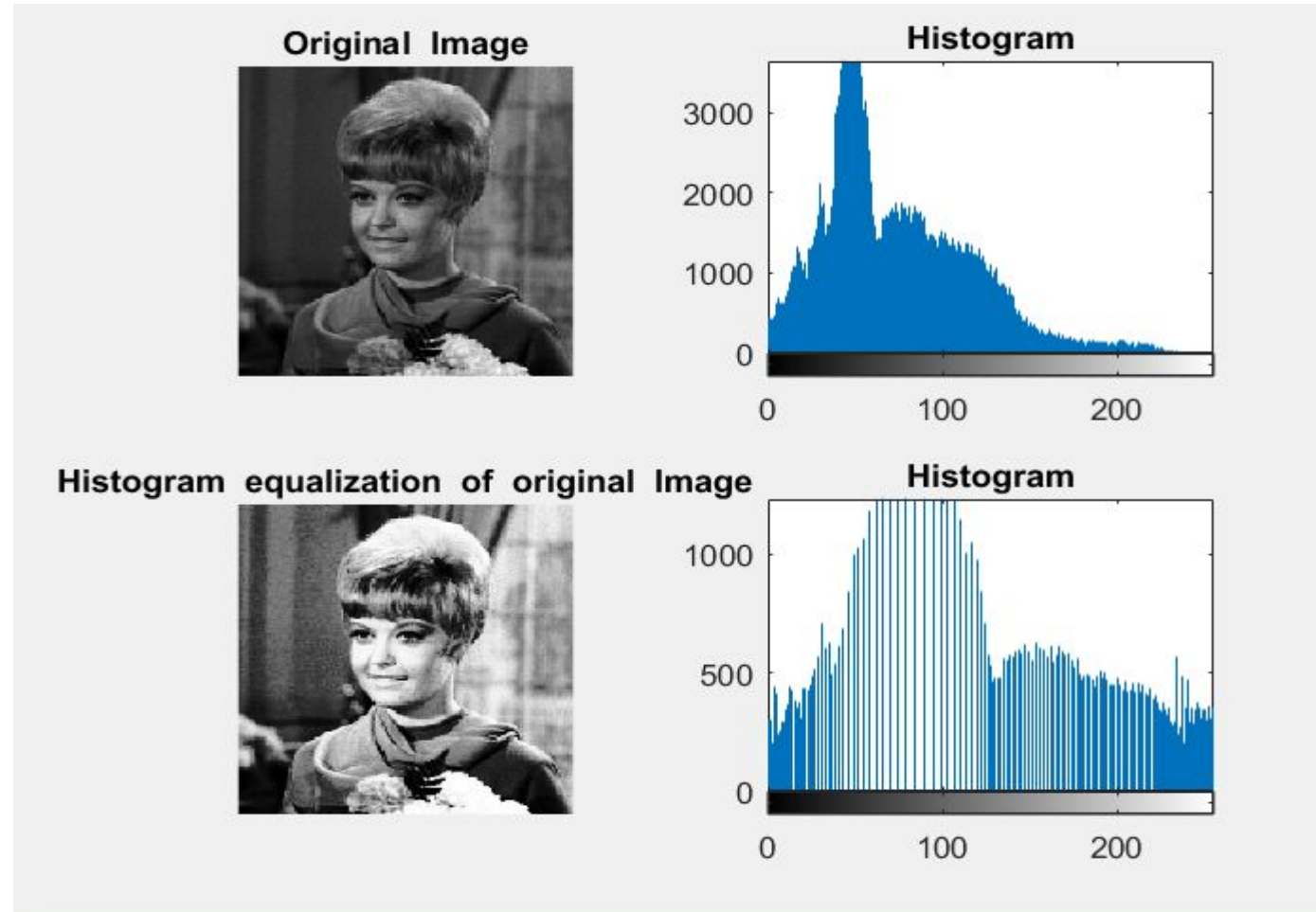
$$C(i) = \sum_{j=0}^i p(j)$$

Normalize the CDF and transform the image using the normalized CDF.

Map the new pixel values to form the output image

$$I' = (L - 1) \cdot C(I)$$

Histogram Equalisation



Histogram Equalisation-Numerical Example

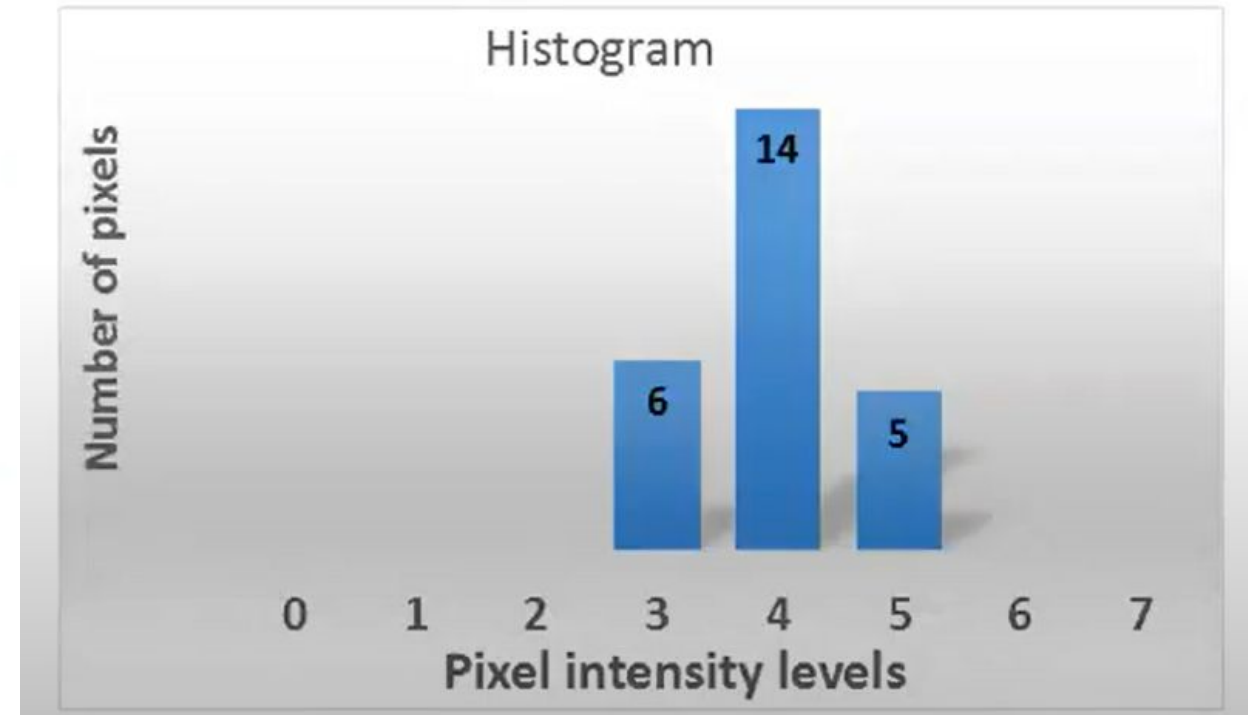
- Example:

$$f(x, y) =$$

4	4	4	4	4
3	4	5	4	3
3	5	5	5	3
3	4	5	4	3
4	4	4	4	4

Histogram Equalisation-Numerical Example

Gray level	0	1	2	3	4	5	6	7
No. of pixel nk	0	0	0	6	14	5	0	0





Gray level	No. of pixel = nk	PDF = $\frac{\text{No. of pixel}}{\text{sum}}$ $P_k = \frac{nk}{N}$	(Running sum) CDF S_k	Running sum * maximum gray level $7 \times s_k$	Histogram equalization level
0	0	$0/25 = 0$	$0 \rightarrow 0$	$7 \times 0 = 0$	0
1	0	$0/25 = 0$	$0 \rightarrow 0$	$7 \times 0 = 0$	0
2	0	$0/25 = 0$	$0 \rightarrow 0$	$7 \times 0 = 0$	0
3	6	$6/25 = 0.24$	$0.24 \rightarrow 0.24$	$7 \times 0.24 = 1.68$	2
4	14	$14/25 = 0.56$	$0.8 \rightarrow 0.8$	$7 \times 0.8 = 5.6$	6
5	5	$5/25 = 0.2$	$1 \rightarrow 1$	$7 \times 1 = 7$	7
6	0	$0/25 = 0$	$1 \rightarrow 1$	$7 \times 1 = 7$	7
7	0	$0/25 = 0$	$1 \rightarrow 1$	$7 \times 1 = 7$	7
N= 25					

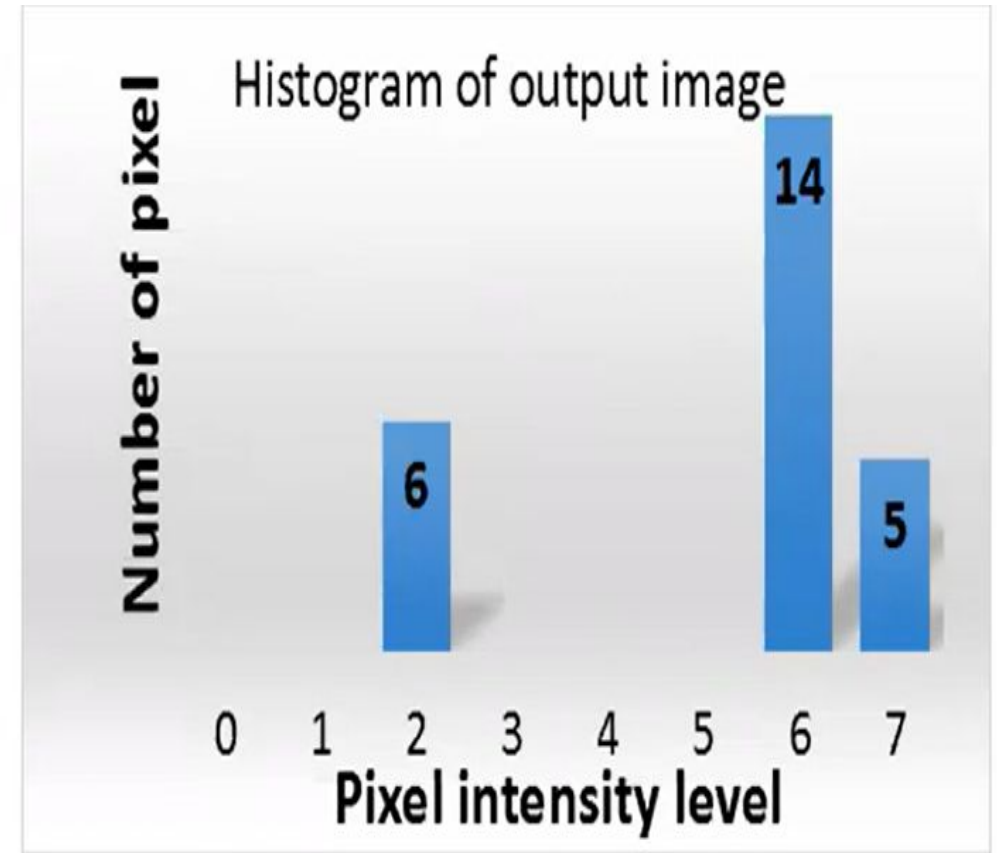
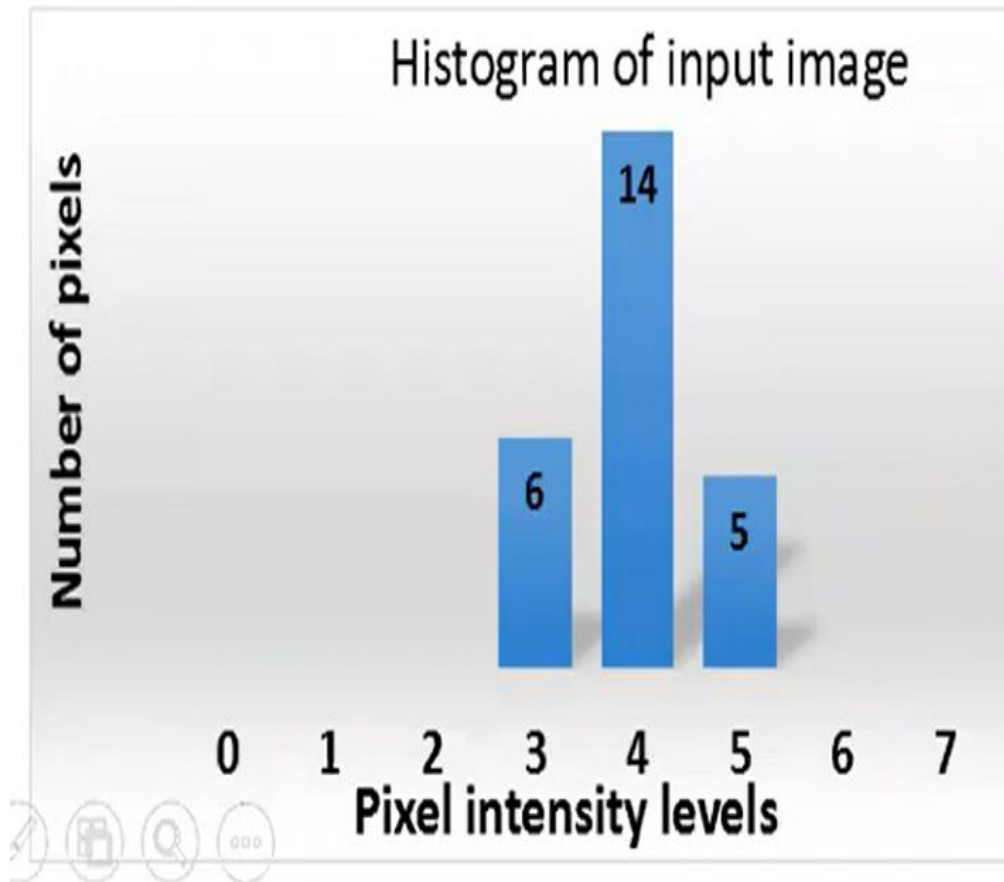
4	4	4	4	4
3	4	5	4	3
3	5	5	5	3
3	4	5	4	3
4	4	4	4	4



6	6	6	6	6
2	6	7	6	2
2	7	7	7	2
2	6	7	6	2
6	6	6	6	6

Gray level	0	1	2	3	4	5	6	7
No. of pixel nk	0	0	0	6	14	5	0	0

Gray level	0	1	2	3	4	5	6	7
No. of pixel nk	0	0	6	0	0	0	14	5



- Histogram Equalisation :

<https://colab.research.google.com/drive/1QzJdQ-hNXVf9asuVmmud6RArkMLkKn2w?usp=sharing>

Thresholding techniques

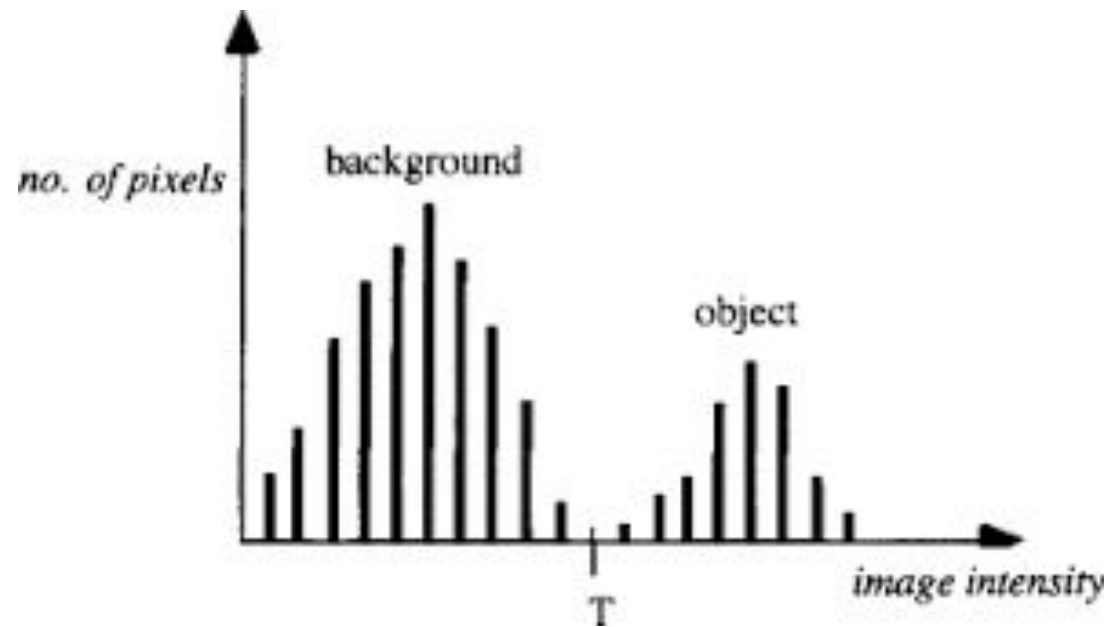
- Thresholding is a fundamental technique in image processing used to **separate objects** of interest from the background.
- By applying a threshold value, pixel intensities are **classified into different categories**, simplifying analysis.

Thresholding techniques

- **Global Thresholding**
- **Adaptive Thresholding**
- **Otsu's Thresholding**

Thresholding techniques

- **Global Thresholding**
- A single threshold value is chosen and applied to the entire image.



Compare each pixel value $I(x, y)$ with the threshold T :

$$I'(x, y) = \begin{cases} 255 & \text{if } I(x, y) \geq T \\ 0 & \text{if } I(x, y) < T \end{cases}$$

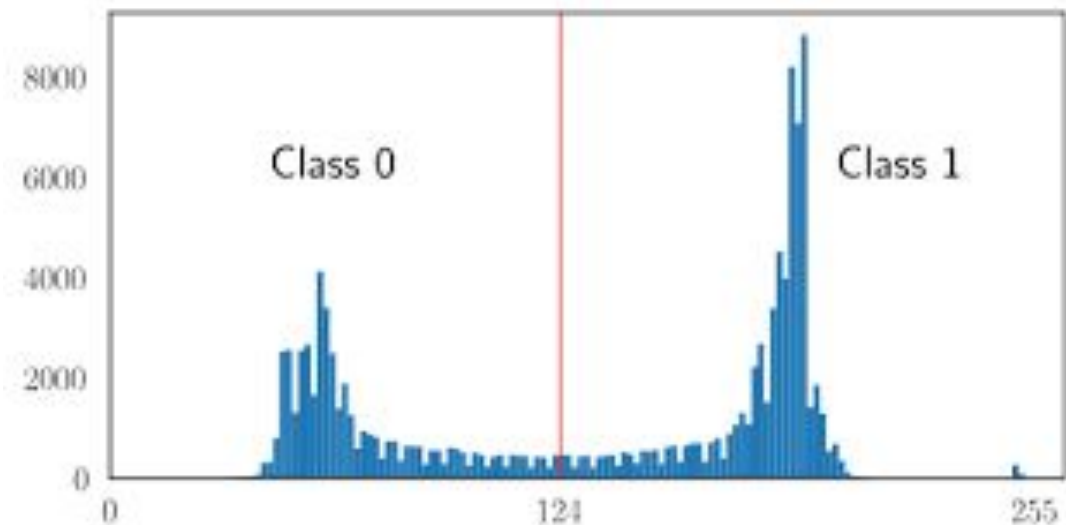
● Adaptive Thresholding

- The threshold value T is computed locally for small regions of the image.
 - **Mean Thresholding:** T is the mean of the neighborhood pixel intensities.
 - **Gaussian Thresholding:** T is a weighted sum of the neighborhood intensities with a Gaussian kernel.



Otsu's Thresholding

- An automatic, global thresholding method that finds T to minimize intra-class variance (or maximize inter-class variance) between background and foreground.
- Ideal for bimodal histograms.
- Implementation:
- Calculate the histogram and probabilities of each intensity level.
- Iterate through potential thresholds to compute the variance for background and foreground and select the one minimizing intra-class variance.



Scale-Invariant Feature Transform (SIFT)

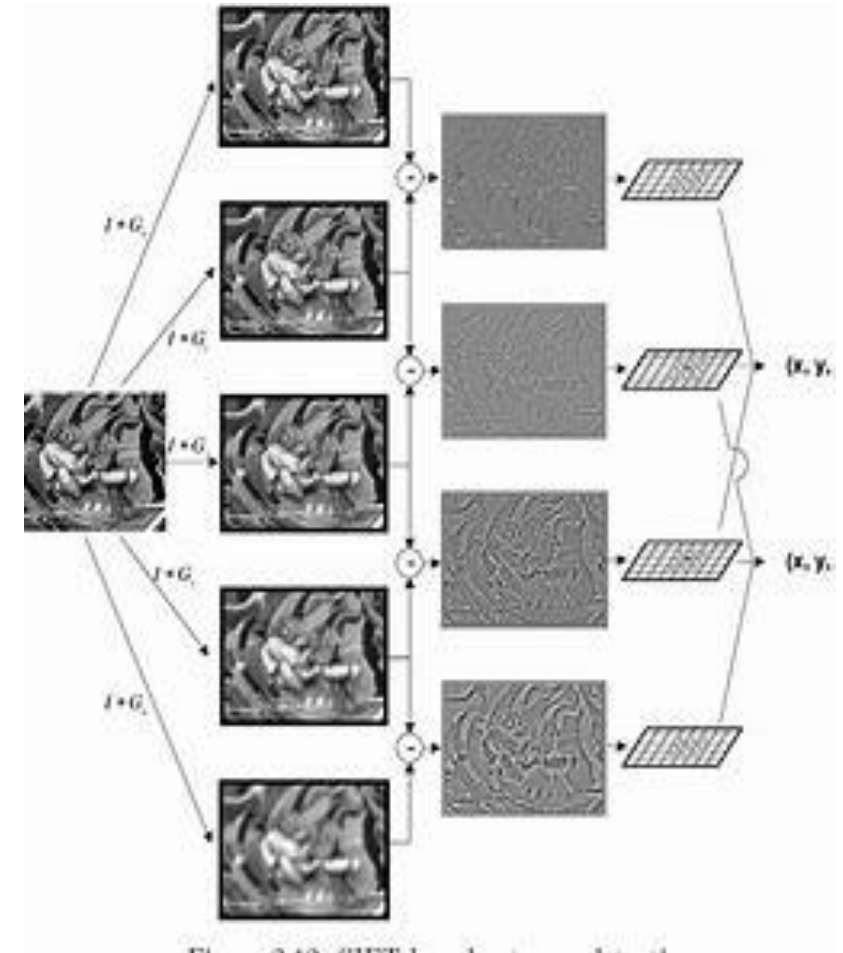
- SIFT is a robust algorithm designed to identify and describe local features in images that are invariant to scale, rotation, and partially invariant to affine transformations and illumination changes.
- This means that SIFT can detect the same features in an image even if the image is resized, rotated, or viewed under different lighting conditions.
- This property makes SIFT extremely valuable for tasks that require matching points between different views of the same scene or object.

Scale-Invariant Feature Transform (SIFT)

- Scale-Space Extrema Detection
- Keypoint Localization
- Orientation Assignment
- Keypoint Descriptor

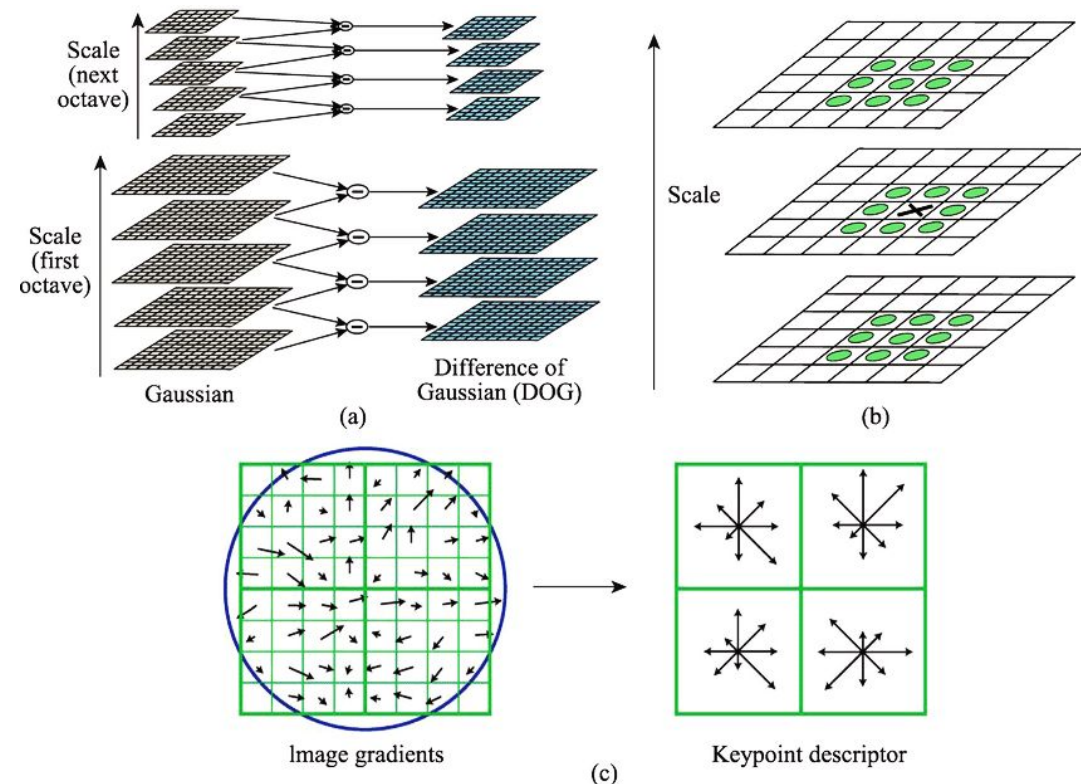
Scale-Invariant Feature Transform (SIFT)

- **Scale-Space Extrema Detection**
- The first step involves identifying key points that are invariant to scale.
- This is achieved by constructing a scale-space representation of the image using a Gaussian function.
- The image is progressively blurred with Gaussian filters of increasing standard deviation, creating a series of images known as the scale space.
- The Difference of Gaussians (DoG) is then computed by subtracting adjacent Gaussian-blurred images. Local extrema in the DoG images are detected, which correspond to potential key points.



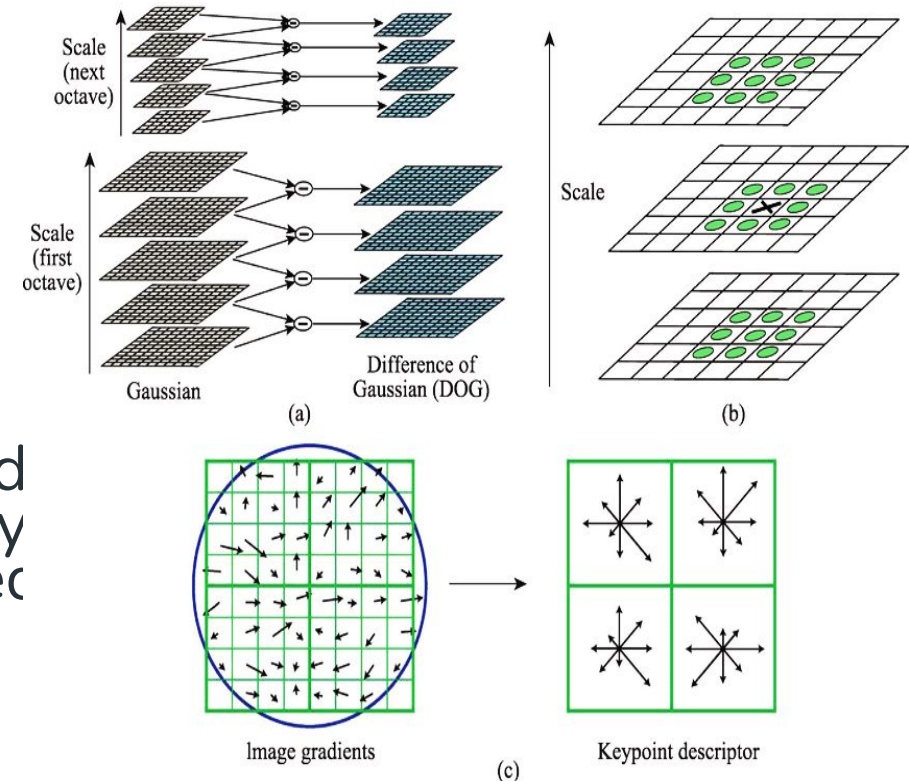
Scale-Invariant Feature Transform (SIFT)

- **Keypoint Localization**
- Keypoint localization assigns orientations to the keypoints detected in the scale-space images.
- The purpose is to eliminate low contrast, unstable, and edge keypoints, and to make the keypoints robust to rotation and affine distortion.
- This is done by calculating the local image gradients and the Laplacian around each keypoint, and finding the dominant orientation(s) in that region



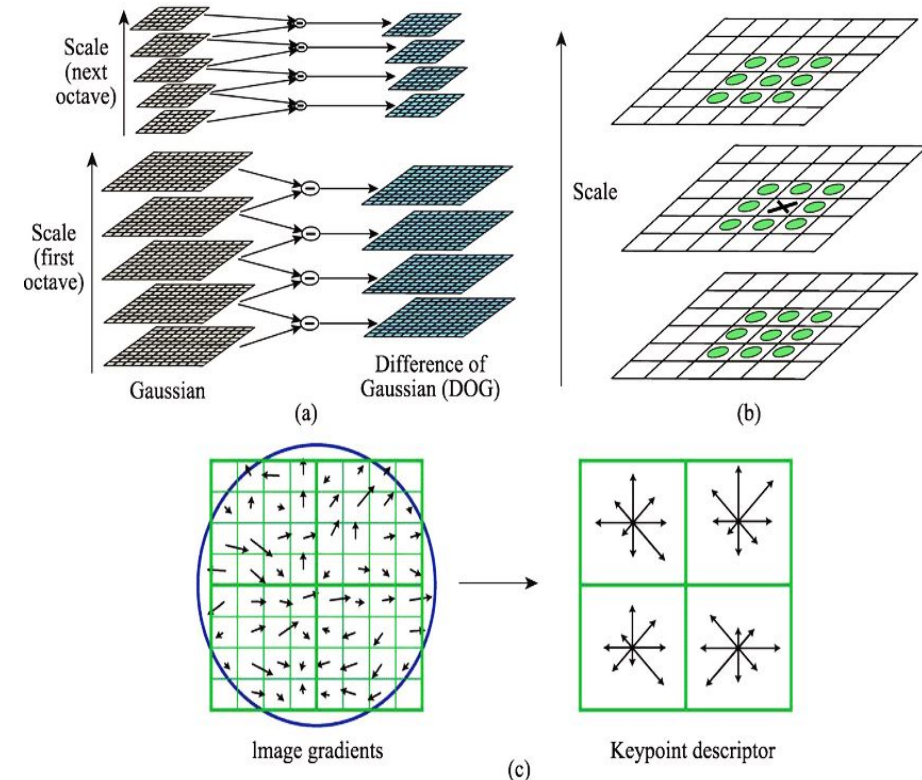
Scale-Invariant Feature Transform (SIFT)

- **Orientation Assignment**
- Each key point is assigned one or more orientations based on the local image gradient directions.
- This step ensures that the key point descriptors are invariant to image rotation.
- The dominant gradient direction is identified within a local neighborhood around each key point, and an orientation histogram is created
- The peak(s) of this histogram represent the assigned orientation(s).



Scale-Invariant Feature Transform (SIFT)

- **Keypoint Descriptor**
- The final step is to create a descriptor for each key point.
- This descriptor is a vector that describes the local image region around the key point.
- The image gradient magnitudes and orientations are sampled within a local region around the key point, and these values are used to create a 128-dimensional vector.
- The descriptor is normalized to reduce the effects of illumination changes.



Applications of SIFT

- Object Recognition
- Image Stitching
- 3D Reconstruction